

VulnTriage-RAG

Exploit-aware dependency vulnerability triage with hybrid RAG, cited verdicts and a retrieval benchmark

One-day build spec for Claude Code | Python / PyPI scope | Prepared 27 Sep 2026

What you will have at the end of the day. A CLI and a FastAPI service that take a `requirements.txt`, find every known vulnerability in it, rank each one by real-world exploit risk (CISA KEV + FIRST EPSS + CVSS), compute the minimum safe upgrade, and write a short verdict whose every claim is cited to an OSV/CVE/KEV/EPSS record. Plus an `/ask` endpoint for natural-language questions over your scan, and an **evaluation report** comparing four retrieval setups (BM25, vector, hybrid, hybrid + reranker) with Recall@k and MRR. That table is the part that makes this a RAG project and not a wrapper.

Why this project for you. It hits the Keysight K3P JD almost word for word ("triage security scan output - CVEs, SBOM completeness and license findings", "LLM use cases for metadata enrichment"), the HFM AI/ML JD ("RAG components: document processing, chunking, and vector search", "experiments comparing ... retrieval strategies", "production Python codebase, including tests") and the SDET JD (RAG, agents, CI/CD integration). It reuses your existing stack: FastAPI, PostgreSQL, Docker, Ollama, pytest.

Core design rule (say this in interviews): *code computes, the LLM explains*. Version matching, minimum safe version and priority are deterministic Python. The LLM only writes the summary and rationale, and a validator rejects any citation that is not in the evidence it was given.

Contents

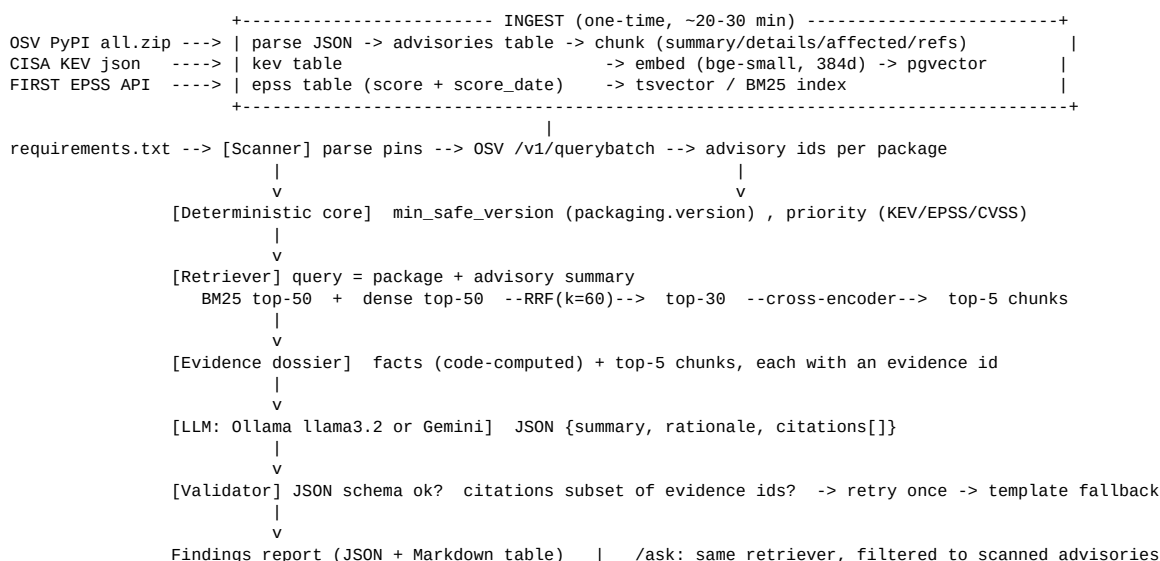
- 1. Scope: what is in and out for one day
- 2. Architecture
- 3. Tech stack and setup
- 4. Data sources (exact URLs)
- 5. Database schema
- 6. Ingestion, chunking and embeddings
- 7. Scanner, minimum safe version and priority
- 8. Retrieval pipeline (BM25, vector, hybrid RRF, reranker)
- 9. Verdict generation and citation validation
- 10. `/ask` endpoint
- 11. Evaluation harness
- 12. API and CLI
- 13. Repo layout, CLAUDE.md and tests
- 14. Hour-by-hour plan with Claude Code prompts
- 15. Cut list, pitfalls, README, resume and interview notes

1. Scope for one day

IN (must ship today)	OUT (stretch / later)
PyPI ecosystem only; input is a requirements.txt with pinned versions (==)	npm, Maven, Go; unpinned ranges; lockfile parsing (poetry.lock, package-lock.json)
OSV PyPI bulk dump as the advisory corpus (local Postgres)	NVD API sync (rate-limited; OSV already carries CVE aliases and CVSS vectors)
CISA KEV feed + FIRST EPSS API for exploit signals	Training a KEV-prediction classifier vs EPSS
Hybrid retrieval: BM25 + dense (pgvector) + RRF + cross-encoder reranker	Fine-tuning embedders or rerankers
Deterministic min-safe-version and priority; LLM verdict with validated citations	Function-level reachability (tree-sitter call graphs)
Eval harness: 4 retrieval setups x Recall@1/5/10 + MRR@10 + latency; generation checks	TREC-style nugget coverage, human eval
FastAPI (/scan, /ask, /advisory/{id}) + CLI + pytest suite + README with results table	GitHub Action, Streamlit UI, cloud deploy (do only if time remains)

Honest framing: this version tells you a vulnerable version is *installed*, not that the vulnerable code path is *reachable*. Say so in the README. Reachability is the named next step.

2. Architecture



3. Tech stack and setup

Layer	Choice	Why
Language	Python 3.11+, uv or venv	Your main language
API	FastAPI + Pydantic v2	Used in SynthSales
DB	PostgreSQL 16 + pgvector (Docker image pgvector/pgvector:pg16)	One store for rows, vectors and full-text
DB access	psycopg 3 (plain SQL) + pgvector python adapter	Faster to write than ORM models today
Embeddings	sentence-transformers, BAAI/bge-small-en-v1.5 (384-d, CPU ok)	Small, strong, fast on a laptop
Sparse	rank_bm25 (BM25Okapi) held in memory, pickled to disk	True BM25; Postgres ts_rank is not BM25
Reranker	cross-encoder/ms-marco-MiniLM-L-6-v2	Standard, CPU-friendly

Layer	Choice	Why
LLM	Ollama llama3.2:3b with format=json (default); Gemini Flash as option	You have used both
Versions	packaging (Version, parse), cvss (CVSS3 base score from vector)	Correct PEP 440 comparisons
HTTP	httpx	Async + retries
CLI	typer + rich	Nice terminal tables for the demo
Tests	pytest, respx (mock HTTP)	No network in unit tests

docker-compose.yml

```
services:
  db:
    image: pgvector/pgvector:pg16
    environment:
      POSTGRES_USER: vt
      POSTGRES_PASSWORD: vt
      POSTGRES_DB: vulntrriage
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

requirements / pyproject deps

```
fastapi uvicorn[standard] pydantic pydantic-settings psycpg[binary] pgvector
sentence-transformers rank_bm25 httpx typer rich packaging cvss
ollama google-genai          # pick one LLM client; keep both behind an interface
pytest respx
```

.env

```
DATABASE_URL=postgresql://vt:vt@localhost:5432/vulntrriage
LLM_PROVIDER=ollama          # or gemini
OLLAMA_MODEL=llama3.2:3b
GEMINI_API_KEY=              # only if LLM_PROVIDER=gemini; never commit
EMBED_MODEL=BAAI/bge-small-en-v1.5
RERANK_MODEL=cross-encoder/ms-marco-MiniLM-L-6-v2
```

4. Data sources

Source	URL / call	Notes
OSV PyPI bulk dump	GET https://osv-vulnerabilities.storage.googleapis.com/PyPI/all.zip	Zip of one JSON file per advisory (PYSEC-*, GHSA-*, etc.). No key, no rate limit. This is your corpus.
OSV batch query	POST https://api.osv.dev/v1/querybatch body: {"queries":[{"package":{"name":"jinja2","ecosystem":"PyPI"},"version":"2.10"}]}	Returns vuln ids per query (in the same order). Use it for exact version matching so you do not reimplement OSV range semantics. Look ids up in your local table; fall back to GET /v1/vulns/{id} if missing.
CISA KEV	GET https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json	Field 'vulnerabilities': cveID, vendorProject, product, dateAdded, dueDate, knownRansomwareCampaignUse.
FIRST EPSS	GET https://api.first.org/data/v1/epss?cve=CVE-A,CVE-B,...	Batch comma-separated CVEs (keep batches ~50-100). Returns epss, percentile, date. Store the date: EPSS changes daily.

Expect few KEV hits on PyPI. KEV is dominated by vendor products (Microsoft, Cisco, Apache Log4j...). That is realistic, not a bug. Unit-test the P0 path with a fixture, and mention in the README that npm/Maven support would surface more KEV matches.

OSV record fields you need

```
id, aliases[]          -> CVE ids are the aliases starting with "CVE-"
```

```

summary, details          -> text for chunks
published, modified
severity[]                -> {"type": "CVSS_V3", "score": "CVSS:3.1/AV:N/AC:L/..."}
                           (a vector string, not a number)
affected[]                -> package{name, ecosystem}, versions[],
                           ranges[{type:"ECOSYSTEM", events:[{introduced},{fixed}]}]
references[]              -> {type: "FIX" | "ADVISORY" | "WEB" | "PACKAGE", url}
database_specific.cwe_ids (GHS records)

```

5. Database schema

```

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE advisories (
  id          TEXT PRIMARY KEY,          -- GHSA-xxxx / PYSEC-xxxx
  aliases     TEXT[] NOT NULL DEFAULT '{}',
  cve_ids     TEXT[] NOT NULL DEFAULT '{}',
  package     TEXT NOT NULL,             -- normalized: lower, '_' and '.' -> '-'
  group_id    TEXT,                      -- alias group (union-find over aliases)
  summary     TEXT,
  details     TEXT,
  cvss_vector TEXT,
  cvss_score  REAL,                      -- computed with the cvss lib; NULL if v4/unknown
  cwe_ids     TEXT[] NOT NULL DEFAULT '{}',
  fixed_versions TEXT[] NOT NULL DEFAULT '{}', -- all 'fixed' events for this package
  affected    JSONB,
  refs        JSONB,
  published   TIMESTAMPTZ,
  modified    TIMESTAMPTZ
);
CREATE INDEX ON advisories (package);
CREATE INDEX ON advisories USING GIN (cve_ids);
CREATE INDEX ON advisories USING GIN (aliases);

CREATE TABLE chunks (
  id          TEXT PRIMARY KEY,          -- f"{advisory_id}#{kind}#{n}"
  advisory_id TEXT REFERENCES advisories(id) ON DELETE CASCADE,
  kind        TEXT NOT NULL,             -- summary | details | affected | refs
  header      TEXT NOT NULL,             -- contextual header (see section 6)
  content     TEXT NOT NULL,
  embedding   vector(384)
);
CREATE INDEX ON chunks USING hnsu (embedding vector_cosine_ops);

CREATE TABLE kev (cve_id TEXT PRIMARY KEY, vendor TEXT, product TEXT, date_added DATE,
  due_date DATE, ransomware TEXT, notes TEXT);
CREATE TABLE epss (cve_id TEXT PRIMARY KEY, epss REAL, percentile REAL, score_date DATE,
  fetched_at TIMESTAMPTZ DEFAULT now());

```

6. Ingestion, chunking and embeddings

6.1 Ingest OSV (vt ingest osv)

- Download all.zip to data/raw/ (skip if present and younger than 24h). Iterate entries with zipfile; json.loads each.
- Keep only records with an affected[] entry whose ecosystem is PyPI. One advisory row per record (use the first PyPI package; log records that list several packages).
- Normalize package names (PEP 503): `re.sub(r'[-_.]+', '-', name).lower()`. Do the same to the user's requirements.
- `cve_ids` = [a for a in aliases if a.startswith('CVE-')] (+ id itself if it is a CVE).
- `cvss_score`: for severity entries of type CVSS_V3, `CVSS3(vector).base_score`; wrap in try/except, leave NULL otherwise.
- `fixed_versions`: every events[].fixed across ECOSYSTEM ranges for that package.
- Upsert with `INSERT ... ON CONFLICT (id) DO UPDATE`, in batches of 1,000 (executemany). Print counts.
- Add `--limit N` for quick dev runs (e.g. 2,000 advisories) before the full load.

6.2 Ingest KEV and EPSS (vt ingest kev / vt ingest epss)

- KEV: one GET, upsert all rows.
- EPSS: collect DISTINCT unnest(cve_ids) from advisories. Full corpus can be ~10k+ CVEs; at 100 per call that is 100+ calls, so add a 0.2-0.5 s sleep and a retry with backoff. Also expose epss_for(cves) used lazily by the scanner so a scan never waits on a full refresh.

6.3 Chunking

One advisory becomes up to four structure-aware chunks. Every chunk gets a **contextual header** prepended before embedding and BM25 indexing (the cheap version of Anthropic-style contextual retrieval):

```
header = (f"[PyPI:{package}] [{advisory_id}] "
          f"[{'', '.join(cve_ids) or 'no-CVE'}] [CWE:{', '.join(cwe_ids) or '-'}] [{kind}]")

summary : summary text (always)
details : details text; if > ~1,200 chars, split on paragraphs into ~800-char
          pieces with 1 paragraph overlap
affected : "Affected: introduced 2.0, fixed 2.11.3; introduced 3.0, fixed 3.0.1" (rendered from ranges)
refs      : "Fix commit: <url> ; Advisory: <url>" (FIX and ADVISORY refs only)
```

6.4 Embeddings and BM25 (vt index)

- bge-small: embed header + '\n' + content with normalize_embeddings=True, batch_size 64. For queries, prefix with 'Represent this sentence for searching relevant passages: ' (bge query instruction).
- Write vectors with pgvector's register_vector + executemany in batches.
- BM25: tokenize (lowercase, split on non-alphanumerics, keep tokens like cve-2023-1234 intact by also adding the raw ids), build BM25Okapi over the same texts, pickle {chunk_ids, bm25} to data/bm25.pkl.
- Expected cost on a laptop CPU: tens of thousands of chunks, roughly 10-25 minutes. Run it in the background while you write the scanner.

7. Scanner, minimum safe version and priority

7.1 Parse requirements

- Accept lines name==version; strip comments, extras (name[extra]) and environment markers. Report unpinned lines as 'skipped: not pinned' rather than guessing.
- POST all pins in one /v1/querybatch call (chunk at 1,000). Map results back by index.

7.2 Minimum safe version (deterministic)

```
from packaging.version import Version, InvalidVersion

def min_safe_version(installed: str, advisories: list[Advisory]) -> str | None:
    """Smallest version >= installed that fixes EVERY advisory affecting this package."""
    inst = Version(installed)
    needed = []
    for adv in advisories:
        fixes = sorted(v for v in map(safe_parse, adv.fixed_versions) if v and v > inst)
        if not fixes:
            return None # an advisory has no fix above installed -> "no fix available"
        needed.append(fixes[0]) # earliest fix for this advisory on our upgrade path
    return str(max(needed)) if needed else None
```

Caveat to document: a fix in an older branch (e.g. 3.2.19) and a newer branch (4.1.x) both appear; taking the smallest fix above installed per advisory, then the max across advisories, is a sound and explainable heuristic. Cross-check the final answer with one more querybatch call on the suggested version (it must return zero vulns); record verified: true/false. That check is cheap and makes the claim testable.

7.3 Priority (deterministic, documented)

Priority	Rule (first match wins)	Meaning
P0	Any CVE of the advisory is in KEV	Exploited in the wild. Patch now.

Priority	Rule (first match wins)	Meaning
P1	EPSS >= 0.10 or CVSS >= 9.0	High likelihood or critical severity
P2	CVSS >= 7.0 or EPSS percentile >= 0.90	High
P3	Everything else (incl. no CVSS / no EPSS)	Routine; batch with next upgrade

Thresholds are a documented policy, not a model. Put them in config. The stretch goal (a calibrated classifier that predicts KEV addition) would replace this table, which is a good interview story.

8. Retrieval pipeline

Implement four retrievers behind one interface so the evaluation can swap them:

```
class Retriever(Protocol):
    def search(self, query: str, k: int = 10, advisory_filter: set[str] | None = None) -> list[Hit]: ...
    # Hit = (chunk_id, advisory_id, score, text)

BM25Retriever      -> bm25.get_scores(tokens), argpartition top-k
DenseRetriever     -> SELECT id, advisory_id, 1 - (embedding <=> %s) AS score FROM chunks
                    [WHERE advisory_id = ANY(%s)] ORDER BY embedding <=> %s LIMIT %s
HybridRetriever    -> BM25 top-50 + Dense top-50, Reciprocal Rank Fusion:
                    score(d) = sum over lists of 1 / (60 + rank(d))
RerankedRetriever  -> Hybrid top-30 -> CrossEncoder.predict([(query, text), ...]) -> top-k
```

- **Dedupe to advisory level** for evaluation: several chunks from one advisory should count once (keep the best-ranked chunk).
- **Alias awareness:** the same bug often exists as both PYSEC-* and GHSA-*. Treat two advisories as equivalent if they share any alias. Build an alias-group map at ingest (union-find over aliases) and store group_id. This one detail prevents your metrics from being wrong.
- advisory_filter restricts /ask to advisories found in the current scan (metadata filtering).
- Log latency per stage; report p50 in the eval table.

9. Verdict generation and citation validation

9.1 Evidence dossier (built in code)

```
EVIDENCE (cite ids exactly as written in square brackets)
[FACT:pkg]      package=jinja2 installed=2.10 min_safe_version=3.1.4 (verified=true)
[FACT:prio]     priority=P2 (rule: CVSS >= 7.0)
[GHSA-xxxx]     advisory id; CVEs: CVE-2019-10906
[KEV:CVE-...]   (only present if in KEV) dateAdded=..., ransomware=...
[EPSS:CVE-2019-10906@2026-09-27] epss=0.0123 percentile=0.71
[GHSA-xxxx#summary#0] <chunk text>
[GHSA-xxxx#details#0] <chunk text>
... (top-5 reranked chunks for query = f"{package} {advisory.summary}")
```

9.2 Prompt

```
SYSTEM: You are a security triage assistant. Use ONLY the evidence provided. Every sentence
must be supported by at least one evidence id. Do not invent versions, scores or dates;
they are already computed in FACT lines. If evidence is insufficient, say so.
Return JSON only: {"summary": str (<= 2 sentences, what the bug is and impact),
                  "rationale": str (<= 3 sentences, why this priority and what to do),
                  "citations": [evidence ids]}

USER: <dossier>
```

9.3 Validator

- Parse with Pydantic. Fail -> retry once with the error message appended.
- Every citation must be in the dossier's evidence id set; the list must be non-empty. Unknown ids -> retry once.
- Still failing -> template fallback built from FACT lines ("jinja2 2.10 is affected by GHSA-... (CVE-...). Priority P2 because CVSS 7.5. Upgrade to 3.1.4.") and mark generated_by: 'template'.

- Final finding = code fields (package, installed, advisory_id, cve_ids, priority, min_safe_version, verified, kev, epss) + LLM fields (summary, rationale, citations) + generated_by.
- Record counters: json_valid_rate, citation_valid_rate, fallback_rate. These go into the eval report.

10. /ask endpoint

Natural-language questions over a scan, e.g. "Which of my dependencies have remote code execution issues?", "Is anything here actively exploited?", "What single upgrade removes the most high-priority findings?"

- Input: scan_id + question. Retrieve with RerankedRetriever filtered to that scan's advisories, top-8.
- Also pass a compact table of the scan's findings (package, advisory, priority, min_safe) as [FACT:*] evidence, so aggregate questions work.
- Same JSON + citation validation as the verdict: {"answer": str, "citations": [...]}.
 - Questions that need arithmetic (counts, 'most') are answered from the findings table by the LLM reading FACT lines; keep the table small (it is one scan).

11. Evaluation harness (the most important hour)

11.1 Build the eval set (vt eval build)

- Sample 150 advisories (fixed seed) with details length > 200 chars and at least one CVE.
- For each, ask the LLM to write one developer question that describes the vulnerability **without** naming the package, advisory id or CVE (e.g. "template rendering in a Python web templating library allows sandbox escape to execute code"). Reject outputs that contain the package name or any id (string check) and regenerate once.
- Gold = the advisory's alias group. Save to data/eval/queries.jsonl: {qid, query, gold_group, advisory_id}.
- Add 20 hand-written queries of your own (realistic phrasing) as a separate small set; report it separately. It guards against the LLM-written queries favouring dense retrieval.

11.2 Retrieval metrics (vt eval retrieval)

```
for setup in [BM25, Dense, Hybrid, Hybrid+Rerank]:
    for q in queries:
        ranked_groups = dedupe([group_of(hit.advisory_id) for hit in setup.search(q.query, k=10)])
        record hit@1, hit@5, hit@10, reciprocal_rank (1/rank of gold within top 10, else 0), latency
    report mean Recall@1, Recall@5, Recall@10, MRR@10, p50 latency ms
```

Output a Markdown table to reports/eval.md and paste it into the README. Fill it with whatever numbers you get. Do not tune thresholds on the same 150 queries you report; if you tune, split 100 dev / 50 test.

Setup	Recall@1	Recall@5	Recall@10	MRR@10	p50 ms
BM25					
Dense (bge-small)					
Hybrid (RRF)					
Hybrid + cross-encoder					

11.3 Generation checks (vt eval generation)

- Run verdicts on the demo requirements file (section 15) and 30 random (package, version, advisory) triples.
- Report JSON-valid rate, citation-valid rate, fallback rate, and 'min_safe_version verified' rate.
- Faithfulness spot check: read 10 verdicts yourself; note any unsupported claim. Write the count in the README honestly.

12. API and CLI

Endpoint / command	Behaviour
POST /scan (multipart file or {"requirements": "..."})	Returns scan_id, findings[] sorted by priority then CVSS, and skipped[] (unpinned / unknown).
POST /ask {scan_id, question}	Answer + citations (section 10).
GET /advisory/{id}	Stored advisory with KEV/EPSS enrichment.
GET /health	DB reachable, index loaded, LLM reachable.
vt ingest osv kev epss [--limit N]	Section 6.
vt index	Embeddings + BM25 pickle.
vt scan requirements.txt [--json out.json] [--no-llm]	Rich table in terminal; --no-llm uses template verdicts (fast, useful in CI).
vt ask SCAN_ID "question"	CLI version of /ask.
vt eval build retrieval generation	Section 11; writes reports/eval.md.

Keep scans in memory (dict scan_id -> result) plus write JSON to data/scans/. A scans table is not needed today.

13. Repo layout, CLAUDE.md and tests

```
vulntrriage-rag/  
  CLAUDE.md                # rules for Claude Code (below)  
  README.md  
  docker-compose.yml      pyproject.toml  .env.example  .gitignore (data/, .env)  
  sql/schema.sql  
  vt/  
    config.py              # pydantic-settings  
    db.py                 # connection, register_vector  
    ingest/osv.py  kev.py  epss.py  
    chunking.py  embed.py  bm25_index.py  
    retrieval/base.py  bm25.py  dense.py  hybrid.py  rerank.py  aliases.py  
    scanner/requirements.py  osv_client.py  versions.py  priority.py  
    llm/provider.py (ollama|gemini)  prompts.py  verdict.py  validate.py  
    ask.py  
    api.py                # FastAPI app  
    cli.py                # typer app, entry point "vt"  
    eval/build.py  retrieval.py  generation.py  report.py  
  tests/  
    fixtures/osv_jinja2.json  kev_small.json  epss_small.json  requirements_demo.txt  
    test_requirements_parse.py  test_versions.py  test_priority.py  test_chunking.py  
    test_rrf.py  test_validate.py  test_aliases.py  test_api_scan.py (respx-mocked OSV)  
  data/  reports/
```

CLAUDE.md (paste this into the repo first)

```
# VulnTriage-RAG  
Python 3.11, FastAPI, Postgres+pgvector (docker compose), psycpg3 plain SQL, sentence-transformers.  
Rules:  
- Code computes, LLM explains. Never let the LLM produce versions, scores, dates or priority.  
- Every LLM output is JSON validated by Pydantic; citations must be a subset of provided evidence ids.  
- Unit tests never hit the network; mock HTTP with respx, use tests/fixtures.  
- Normalize package names with PEP 503 everywhere.  
- Treat advisories sharing any alias as the same vulnerability (alias groups) in retrieval and eval.  
- Keep functions small and typed. Run `pytest -q` after each change and fix failures before moving on.  
- Secrets only via .env; never print or commit keys.
```

Minimum tests (write them as you go, not at the end)

- requirements parser: pins, extras, markers, comments, unpinned -> skipped.
- min_safe_version: single advisory; two advisories needing different fixes -> max; advisory with no fix -> None; pre-release and invalid versions.
- priority: KEV beats everything; EPSS and CVSS boundaries (exactly 0.10, 9.0, 7.0); missing data -> P3.
- RRF: hand-computed example with two lists.

- aliases: PYSEC and GHSA sharing a CVE end in one group.
- validator: unknown citation rejected; empty citations rejected; fallback produced.
- API: /scan with mocked OSV returns sorted findings.

14. Hour-by-hour plan with Claude Code prompts

About 11 focused hours. Each block ends with a working, committed state. Start the long-running jobs (OSV ingest, embeddings) early and keep coding while they run. Commit after every block.

Time / block	Prompt to give Claude Code	Done when
0:00-0:30 Setup	Create the repo from section 13. Add CLAUDE.md, docker-compose.yml, pyproject with the deps in section 3, .env.example, .gitignore, sql/schema.sql from section 5, vt/config.py, vt/db.py and a typer CLI with a `vt db init` command that applies schema.sql. Then run docker compose up -d and vt db init.	docker ps shows db; tables exist; `ollama pull llama3.2:3b` started in another terminal.
0:30-2:00 Ingestion	Implement vt/ingest/osv.py, kev.py, epss.py and `vt ingest osv kev epss [--limit N]` exactly as section 6.1-6.2 of the spec: PEP 503 names, cve_ids from aliases, cvss_score via the cvss lib, fixed_versions from ECOSYSTEM ranges, batched upserts, alias groups (union-find over aliases, stored as group_id column). Add tests with fixtures. Run with --limit 2000 first.	Full OSV load running in background; KEV loaded; EPSS loading lazily.
2:00-3:00 Chunking + index	Implement chunking.py (4 kinds + contextual header, section 6.3), embed.py (bge-small, batch 64, normalized, query instruction prefix) and bm25_index.py (pickle). Command `vt index` embeds all chunks without embeddings and rebuilds BM25. Tests for chunking.	`vt index` running in background on the full corpus.
3:00-4:30 Retrievers	Implement the Retriever protocol and BM25, Dense (pgvector cosine, optional advisory filter), Hybrid (RRF k=60, 50+50) and Reranked (cross-encoder on top 30) per section 8, with advisory-level dedupe and alias groups. Add `vt search "query" --setup hybrid_rerank` that prints a rich table. Tests for RRF and dedupe.	vt search returns sensible advisories for 'yaml load arbitrary code execution'.
4:30-5:30 Scanner	Implement scanner/requirements.py, osv_client.py (querybatch with httpx, retries), versions.py (min_safe_version + re-verify via querybatch) and priority.py (section 7 table, thresholds in config). `vt scan FILE --no-llm` prints findings sorted by priority. Tests per section 13.	Demo file scans correctly with template verdicts.
5:30-6:00 Break		
6:00-7:30 Verdicts	Implement llm/provider.py (Ollama with format=json, Gemini optional), prompts.py, verdict.py (dossier per section 9.1, retrieval query = package + advisory summary, top-5 reranked chunks) and validate.py (Pydantic, citation subset check, one retry, template fallback, counters). Wire into vt scan (default LLM on).	Each finding has summary, rationale, valid citations.
7:30-9:00 Evaluation	Implement vt eval build (150 LLM-written queries without package/id leakage, seed 42, plus load data/eval/manual.jsonl), vt eval retrieval (4 setups, Recall@1/5/10, MRR@10, p50 latency, alias-group gold) and vt eval generation (JSON-valid, citation-valid, fallback, verified rates). Write reports/eval.md.	reports/eval.md has real numbers. Write your 20 manual queries while build runs.
9:00-10:00 API + /ask	Implement vt/api.py with POST /scan, POST /ask, GET /advisory/{id}, GET /health per section 12, and vt/ask.py per section 10 (filtered retrieval + findings table as FACT evidence, same validator). Add `vt ask`. Add test_api_scan.py with respx.	curl /scan and /ask work; pytest green.
10:00-11:00 Ship	Write README.md: problem, architecture diagram, quickstart (docker compose, ingest, index, scan), sample output, eval table from reports/eval.md, design decisions (code computes / LLM explains, alias groups, RRF, why BM25+dense), limitations (installed != reachable, PyPI only, few KEV hits), next steps (reachability via tree-sitter, KEV-prediction model, npm). Add a GitHub Actions workflow running pytest.	Pushed to GitHub with a real results table and a 60-second screen recording or GIF of vt scan.

How to drive Claude Code today: start each block with "Read CLAUDE.md and section N of VulnTriage_RAG_1Day_Build_Plan.pdf" (or paste the section). Ask for tests in the same prompt. After each block run pytest yourself, then commit. If a block runs over by 30 minutes, take the cut from section 15 instead of pushing the evaluation later: the eval table is the one thing you must not drop.

15. Cut list, pitfalls, demo, README, resume

If you fall behind, cut in this order

- 1. Gemini provider (keep Ollama only).
- 2. /ask endpoint (keep /scan).
- 3. The FastAPI layer entirely (the CLI is enough to demo).
- 4. Details-chunk splitting (one details chunk per advisory).
- 5. Full corpus: evaluate on a 5,000-advisory subset and say so.
- Never cut: hybrid + reranker, the eval table, validator, tests for versions and priority.

Pitfalls that will cost you an hour if you hit them blind

Pitfall	Fix
PYSEC and GHSA duplicates make Recall look wrong	Alias groups (union-find) at ingest; compare groups, not ids
Package name mismatch (PyYAML vs pyyaml, zope.interface)	PEP 503 normalization on both sides
Version strings like '2.0.0rc1', '1.0-beta', or git hashes in OSV	packaging.Version inside try/except; skip invalid
CVSS given as a vector, not a number; some are CVSS_V4	cvss lib for v3; NULL for others; priority tolerates NULL
LLM returns prose around JSON or invents ids	Ollama format=json, Pydantic, citation subset check, one retry, template fallback
Eval queries leak the package name	String check against package and all ids; regenerate once; drop if still leaking
EPSS calls throttled	Batch 50-100, sleep, backoff; lazy per-scan fetch
Embedding the full corpus is slow	Start it early in the background; --limit for dev
Windows: sentence-transformers / torch install	Install CPU torch wheel first; use Python 3.11 or 3.12

Demo requirements file (old pins with known advisories)

```
django==3.2.0
jinja2==2.10
pyyaml==5.3
requests==2.19.0
urllib3==1.24.1
pillow==8.0.0
flask==0.12
cryptography==3.2
```

README headline structure

- One line: "Is this CVE actually my problem? Exploit-aware triage for Python dependencies, with cited verdicts."
- GIF of vt scan output. Architecture diagram. Quickstart in 5 commands.
- Results: the eval table + generation rates + an honest limitations list.
- Design decisions: code computes / LLM explains; alias groups; RRF; why a cross-encoder.

Resume bullet templates (fill in real numbers only after the eval runs)

Keep bullets as plain text, no bold numbers, same as the rest of your resume. Replace every [] with a measured value from reports/eval.md; delete the clause if you do not have the number.

- Built a dependency vulnerability triage service over [N]k OSV advisories with CISA KEV and FIRST EPSS signals, ranking findings by exploit risk and computing verified minimum safe upgrades.
- Implemented hybrid retrieval (BM25 + bge embeddings in pgvector, reciprocal rank fusion, cross-encoder reranking), raising Recall@5 from [x] (dense only) to [y] on a [150]-query benchmark.
- Designed citation-validated LLM verdicts (Pydantic schema, evidence-id checks, template fallback), reaching [z]% valid citations; FastAPI + CLI, Dockerized Postgres, [N] pytest tests in CI.

Interview talking points

- Why hybrid: advisory text is full of exact tokens (CVE ids, function names, package names) that BM25 catches and dense misses, while paraphrased questions need dense. Show the table.
- Why a reranker: the fused list has the right advisory in the top 30 more often than in the top 5; the cross-encoder fixes ordering at a latency cost you measured.
- Why the LLM does not decide priority: reproducibility and auditability; the policy is a table you can change, and the next step is a calibrated model trained on KEV additions.
- Alias groups: a real data-quality issue you found and fixed; it changed your metrics.
- Limitation you know about: installed is not reachable. Next step: tree-sitter call graph from your entry points to vulnerable symbols named in fix commits.